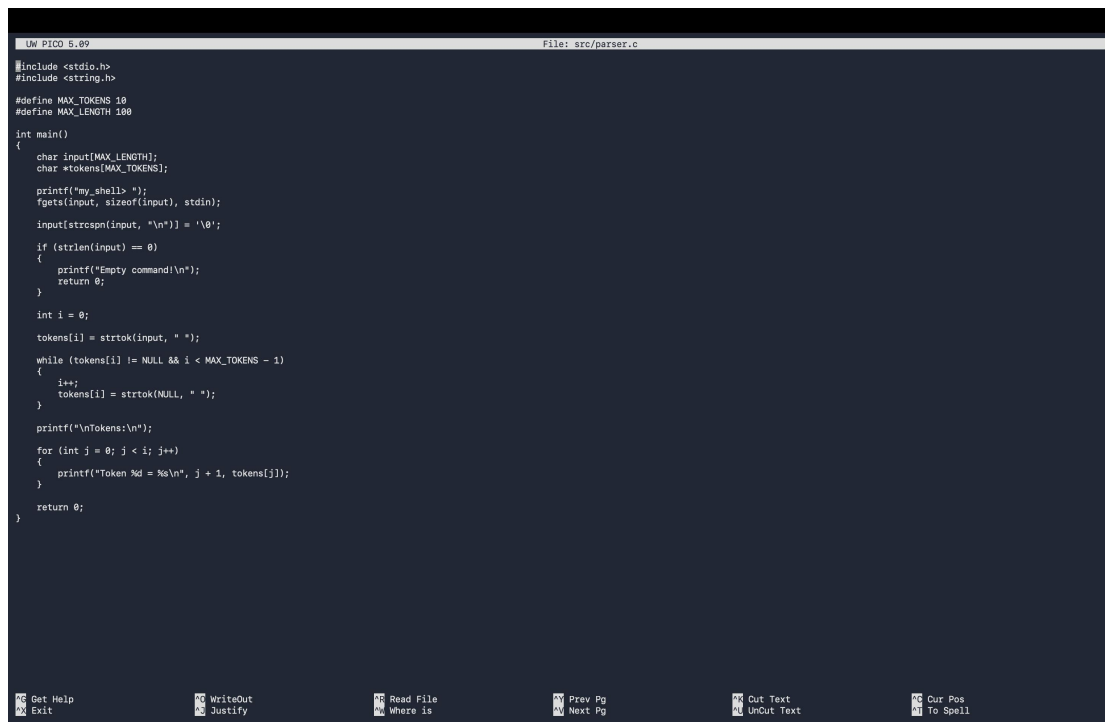


SKILL_WEEK-4

Task 1:

```
Last login: Thu Aug 13 19:56:29 on ttys007
amrutha@Amruthas-MacBook-Air ~ % cd ~/OSSP
[pwd
/Users/amrutha/OSSP
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 1: OSSP project directory



The screenshot shows a PICO editor window titled "File: src/parser.c". The code is a C program for parsing and lexical tokenization. It includes `<stdio.h>` and `<string.h>`. It defines `MAX_TOKENS` as 10 and `MAX_LENGTH` as 100. The `main` function reads input from `stdin`, splits it into tokens using `strtok`, and prints the tokens. The code is as follows:

```
#include <stdio.h>
#include <string.h>

#define MAX_TOKENS 10
#define MAX_LENGTH 100

int main()
{
    char input[MAX_LENGTH];
    char *tokens[MAX_TOKENS];

    printf("my_shell> ");
    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0)
    {
        printf("Empty command!\n");
        return 0;
    }

    int i = 0;
    tokens[i] = strtok(input, " ");
    while (tokens[i] != NULL && i < MAX_TOKENS - 1)
    {
        i++;
        tokens[i] = strtok(NULL, " ");
    }

    printf("\nTokens:\n");
    for (int j = 0; j < i; j++)
    {
        printf("Token %d = %s\n", j + 1, tokens[j]);
    }

    return 0;
}
```

Figure 2: C program for parsing and lexical tokenization

```
amrutha@Amruthas-MacBook-Air OSSP % gcc src/parser.c -o bin/parser
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 3: Successful compilation of parser.c

```
[amrutha@Amruthas-MacBook-Air OSSP % gcc src/parser.c -o bin/parser
[amrutha@Amruthas-MacBook-Air OSSP % ./bin/parser
my_shell> ls -l docs

Tokens:
Token 1 = ls
Token 2 = -l
Token 3 = docs
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 4: Tokenization of the command `ls -l docs`

```
[amrutha@Amruthas-MacBook-Air OSSP % ./bin/parser
my_shell> mkdir project

Tokens:
Token 1 = mkdir
Token 2 = project
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 5: Tokenization of the command `mkdir project`

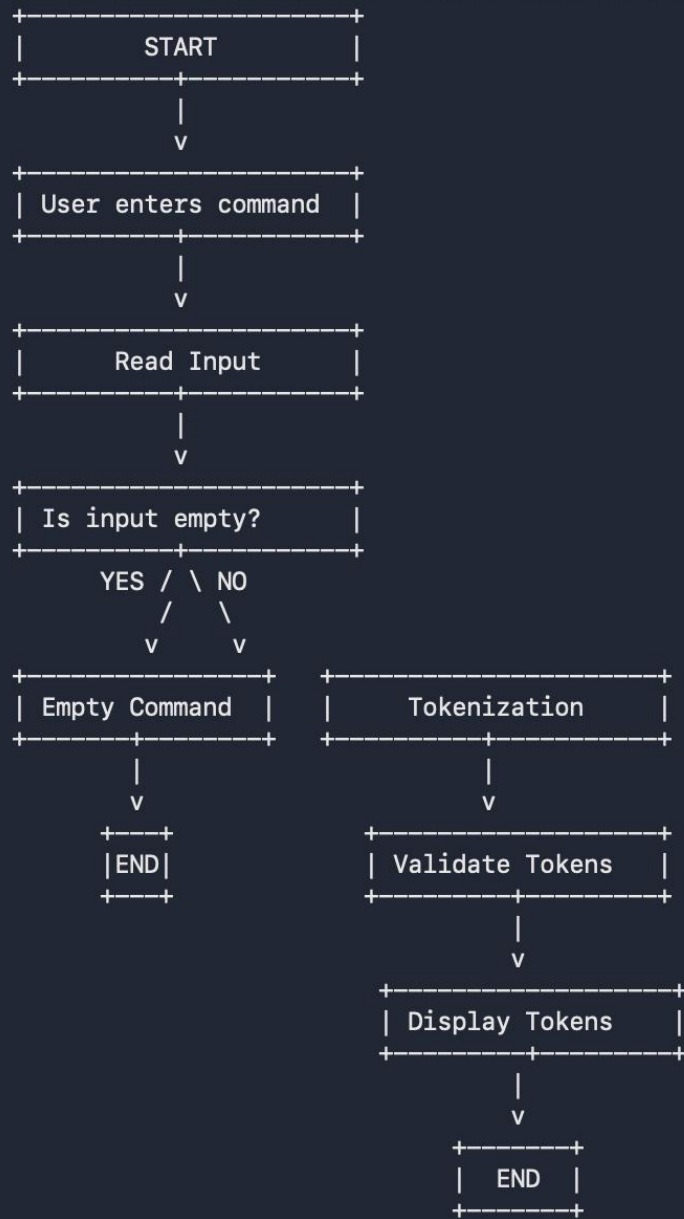
```
[amrutha@Amruthas-MacBook-Air OSSP % ./bin/parser
my_shell>
Empty command!
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 6: Detection and handling of an empty command

```

[amrutha@Amruthas-MacBook-Air OSSP % nano parser_flowchart.txt
[amrutha@Amruthas-MacBook-Air OSSP % cat parser_flowchart.txt

```



```

amrutha@Amruthas-MacBook-Air OSSP % █

```

Figure 7: Flow chart for Parsing and Lexical Analysis

Task 2:

```
amrutha@Amruthas-MacBook-Air ~ % cd ~/OSSP
amrutha@Amruthas-MacBook-Air OSSP % pwd
/Users/amrutha/OSSP
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 1: OSSP project directory for Task 2

A screenshot of a code editor showing a C program named 'quote_parser.c'. The program includes standard headers like <stdio.h>, <stdlib.h>, <string.h>, <unistd.h>, <sys/types.h>, and <sys/wait.h>. It defines constants for MAX_ARGS (20), MAX_INPUT (256), and MAX_TOKEN (256). The main function 'parse_command' takes an input string and an array of arguments. It iterates through the input, handling single quotes and spaces to parse tokens. The tokens are stored in an array, and the program is designed to execute the command. The editor interface shows a dark theme with a status bar at the bottom containing various utility icons and keyboard shortcuts.

```
LM PIDO 5.09 File: src/quote_parser.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_ARGS 20
#define MAX_INPUT 256
#define MAX_TOKEN 256

int parse_command(char *input, char *args[])
{
    static char tokens[MAX_ARGS][MAX_TOKEN];

    int argc = 0;
    int pos = 0;
    int in_single_quote = 0;
    int token_started = 0;

    for (int i = 0; input[i] != '\0'; i++)
    {
        char ch = input[i];

        if (ch == '\\')
        {
            in_single_quote = !in_single_quote;
            token_started = 1;
        }
        else if ((ch == ' ' || ch == '\t') && !in_single_quote)
        {
            if (token_started)
            {
                tokens[argc][pos] = '\0';
                args[argc] = tokens[argc];
                argc++;
                pos = 0;
                token_started = 0;

                if (argc >= MAX_ARGS - 1)
                    break;
            }
        }
        else
        {
            if (pos < MAX_TOKEN - 1)
            {
                tokens[argc][pos++] = ch;
                token_started = 1;
            }
        }
    }

    if (in_single_quote)
    {
        tokens[argc][pos] = '\0';
        args[argc] = tokens[argc];
        argc++;
    }
}
```

Figure 2: C program for single-quote parsing and command execution

```
amrutha@Amruthas-MacBook-Air OSSP % gcc src/quote_parser.c -o bin/quote_parser
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 3: Successful compilation of quote parser

Case 1:(a)

```
amrutha@Amruthas-MacBook-Air OSSP % ./bin/quote_parser
myShell> echo Hello

Parsing Result:
Argument 1: echo
Argument 2: Hello

Child PID: 4395
Hello

Command execution completed.
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 4: Parsing and execution of a normal command

Case 1:(b)

```
amrutha@Amruthas-MacBook-Air OSSP % ./bin/quote_parser
myShell> echo 'Hello World'

Parsing Result:
Argument 1: echo
Argument 2: Hello World

Child PID: 4752
Hello World

Command execution completed.
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 5: Single quotes preserving literal content as one argument

Case 2:

```
amrutha@Amruthas-MacBook-Air OSSP % ./bin/quote_parser
myShell> echo 'Hello World
Syntax Error: Unmatched single quote.
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 6: Detection of unmatched single quote and syntax error

```

amrutha@Amruthas-MacBook-Air OSSP % ./bin/quote_parser
myShell> echo '$HOME'

Parsing Result:
Argument 1: echo
Argument 2: $HOME

Child PID: 5178
$HOME

Command execution completed.
amrutha@Amruthas-MacBook-Air OSSP %

```

Figure 7: Single quotes preserving literal \$HOME

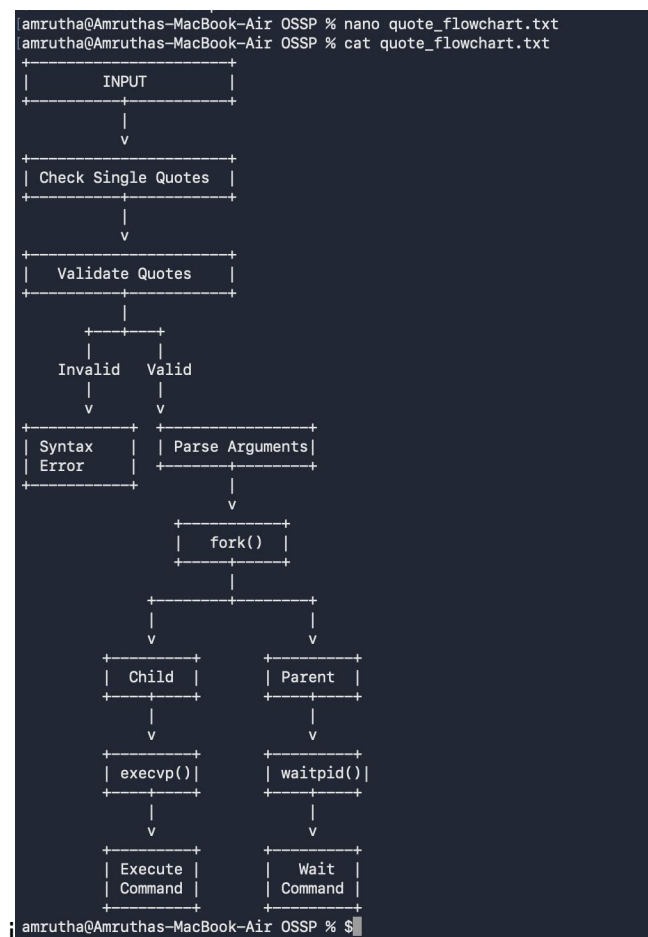


Figure 8: Terminal representation of the flowchart for single-quote parsing and command execution

Task 3:

```
[amrutha@Amruthas-MacBook-Air ~ % cd ~/OSSP  
[amrutha@Amruthas-MacBook-Air OSSP % pwd  
/Users/amrutha/OSSP  
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 1: OSSP project directory for Task 4

```
int parse_command(char *input, char *args[])  
{  
    static char tokens[MAX_ARGS][MAX_TOKEN];  
  
    int argc = 0;  
    int pos = 0;  
    int in_double_quote = 0;  
    int token_started = 0;  
  
    for (int i = 0; input[i] != '\0'; i++)  
    {  
        char ch = input[i];  
  
        if (ch == '"')  
        {  
            in_double_quote = !in_double_quote;  
            token_started = 1;  
        }  
        else if ((ch == ' ' || ch == '\t') && !in_double_quote)  
        {  
            if (token_started)  
            {  
                tokens[argc][pos] = '\0';  
                args[argc] = tokens[argc];  
                argc++;  
  
                pos = 0;  
                token_started = 0;  
  
                if (argc >= MAX_ARGS - 1)  
                    break;  
            }  
            else if (ch == '$')  
            {  
                char variable[100];  
                int v = 0;  
                i++;  
  
                while (input[i] != '\0' &&  
                    ((input[i] == '$' && input[i] <= 'Z') ||  
                     input[i] == '$' && input[i] <= 'z') ||  
                     input[i] == '$' && input[i] <= '0' ||  
                     input[i] == '$' && input[i] <= '9'))  
                {  
                    variable[v++] = input[i];  
                    i++;  
                }  
  
                variable[v] = '\0';  
                i++;  
  
                char *value = getenv(variable);  
  
                if (value != NULL)  
                {  
                    for (int j = 0; value[j] != '\0'; j++)  
                    {  
                        if (pos < MAX_TOKEN - 1)  
                            tokens[argc][pos++] = value[j];  
                    }  
                }  
            }  
        }  
    }  
}
```

Figure 2(a): Double-quote parsing program – input and quote validation

```
    }  
    }  
    token_started = 1;  
}  
else  
{  
    if (pos < MAX_TOKEN - 1)  
        tokens[argc][pos++] = ch;  
  
    token_started = 1;  
}  
}  
  
if (in_double_quote)  
{  
    printf("Syntax Error: Unmatched double quote.\n");  
    return -1;  
}  
  
if (token_started)  
{  
    tokens[argc][pos] = '\0';  
    args[argc] = tokens[argc];  
    argc++;  
}  
  
args[argc] = NULL;  
  
return argc;  
}  
  
int main()  
{  
    char input[MAX_INPUT];  
    char *args[MAX_ARGS];  
  
    printf("myShell> ");  
  
    if (fgets(input, sizeof(input), stdin) == NULL)  
        return 0;  
}
```

Figure 2(b): Double-quote parsing program – variable expansion and tokenization

```

input[strcspn(input, "\n")] = '\0';

if (strlen(input) == 0)
{
    printf("Empty command.\n");
    return 0;
}

int argc = parse_command(input, args);

if (argc < 0)
    return 1;

printf("\n--- Parsed Arguments ---\n");

for (int i = 0; i < argc; i++)
{
    printf("Argument %d: %s\n", i + 1, args[i]);
}

pid_t pid = fork();

if (pid < 0)
{
    perror("fork");
    return 1;
}

if (pid == 0)
{
    printf("\nChild Process\n");
    printf("PID = %d\n\n", getpid());

    execvp(args[0], args);

    perror("execvp");
    exit(1);
}
else
{
    waitpid(pid, NULL, 0);
    printf("\nChild process completed.\n");
}

return 0;
}
amrutha@Amruthas-MacBook-Air OSSP % █

```

Figure 2(c): Double-quote parsing program – process creation and command execution

Case 1:

```
[amrutha@Amruthas-MacBook-Air OSSP % gcc src/double_quote_parser.c -o bin/double_quote_parser
[amrutha@Amruthas-MacBook-Air OSSP % ./bin/double_quote_parser
myShell> echo "Hello $USER"

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello amrutha

Child Process
PID = 18958

Hello amrutha

Child process completed.
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 3: Successful compilation of double-quote parser and Double quotes preserving spaces and expanding the \$USER variable

Case 2:

```
[amrutha@Amruthas-MacBook-Air OSSP % ./bin/double_quote_parser
myShell> echo "Hello World
Syntax Error: Unmatched double quote.
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 4: Detection of unmatched double quote

```
[amrutha@Amruthas-MacBook-Air OSSP % ./bin/double_quote_parser
myShell> echo "Hello World"

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello World

Child Process
PID = 19967

Hello World

Child process completed.
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 5: Double quotes preserving spaces as a single argument

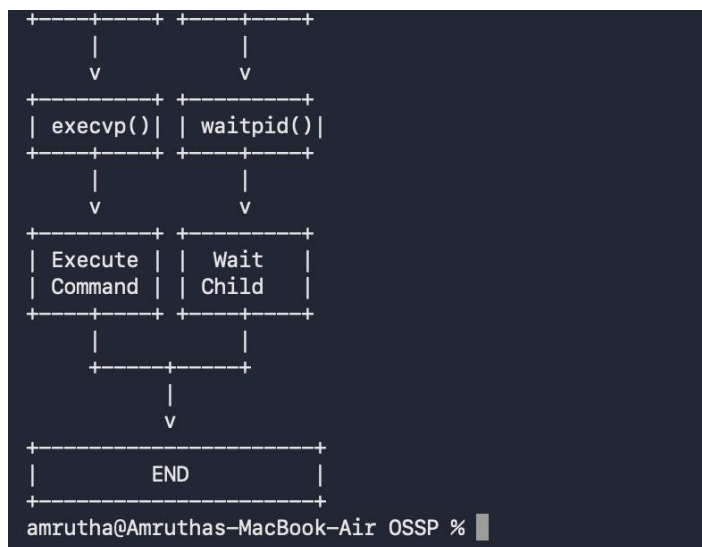
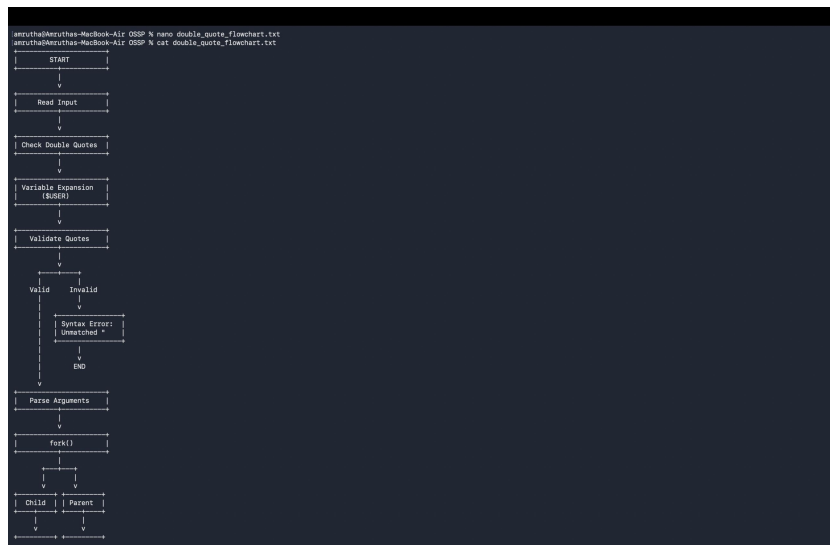


Figure 6: Terminal flowchart for double-quote validation, variable expansion and command execution

```

amrutha@Amruthas-MacBook-Air OSSP % git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   src/parser.c

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        bin/double_quote_parser
        bin/parser
        bin/quote_parser
        double_quote_flowchart.txt
        parser_flowchart.txt
        quote_flowchart.txt
        src/double_quote_parser.c
        src/parser.c.save
        src/quote_parser.c

no changes added to commit (use "git add" and/or "git commit -a")
amrutha@Amruthas-MacBook-Air OSSP %

```

Figure 7: Git status after implementing double-quote parsing

Task 4:

```
[amrutha@Amruthas-MacBook-Air ~ % cd ~/OSSP  
[amrutha@Amruthas-MacBook-Air OSSP % pwd  
/Users/amrutha/OSSP
```

Figure 1: OSSP project directory for Task 4

```
int parse_command(char *input, char *args[])  
{  
    static char tokens[MAX_ARGS][MAX_TOKEN];  
  
    int argc = 0;  
    int pos = 0;  
    int escaped = 0;  
    int token_started = 0;  
  
    for (int i = 0; input[i] != '\0'; i++)  
    {  
        char ch = input[i];  
  
        if (escaped)  
        {  
            if (pos < MAX_TOKEN - 1)  
                tokens[argc][pos++] = ch;  
  
            escaped = 0;  
            token_started = 1;  
        }  
        else if (ch == '\\')  
        {  
            escaped = 1;  
            token_started = 1;  
        }  
        else if (ch == ' ' || ch == '\t')  
        {  
            if (token_started)  
            {  
                tokens[argc][pos] = '\0';  
                args[argc] = tokens[argc];  
                argc++;  
  
                pos = 0;  
                token_started = 0;  
  
                if (argc == MAX_ARGS - 1)  
                    break;  
            }  
            else  
            {  
                if (pos < MAX_TOKEN - 1)  
                    tokens[argc][pos++] = ch;  
  
                token_started = 1;  
            }  
        }  
  
        if (escaped)  
        {  
            printf("Syntax Error: Trailing escape character.\n");  
            return -1;  
        }  
  
        if (token_started)  
        {  
            tokens[argc][pos] = '\0';  
            args[argc] = tokens[argc];  
        }  
    }  
}
```

Figure 2(a): Escape sequence parser – input processing and escape handling

```
    return args;  
}  
  
int main()  
{  
    char input[MAX_INPUT];  
    char *args[MAX_ARGS];  
  
    printf("myShell> ");  
  
    if (!fgets(input, sizeof(input), stdin) || !input[0])  
        return 0;  
  
    input[strcspn(input, "\n")] = '\0';  
  
    if (strlen(input) == 0)  
    {  
        printf("Empty command.\n");  
        return 0;  
    }  
  
    int argc = parse_command(input, args);  
  
    if (argc < 0)  
        return 1;  
  
    printf("\n--- Parsed Arguments ---\n");  
    for (int i = 0; i < argc; i++)  
    {  
        printf("Argument %d: %s\n", i + 1, args[i]);  
    }  
  
    pid_t pid = fork();  
  
    if (pid < 0)  
    {  
        perror("fork");  
        return 1;  
    }  
  
    if (pid == 0)  
    {  
        printf("\nChild Process\n");  
        printf("PID = %d\n", getpid());  
  
        execvp(args[0], args);  
  
        perror("execvp");  
        exit(1);  
    }  
    else  
    {  
        waitpid(pid, NULL, 0);  
        printf("\nChild process completed.\n");  
    }  
  
    return 0;  
}
```

Figure 2(b): Escape sequence parser – tokenization and validation and process creation and command execution

```
amrutha@Amruthas-MacBook-Air OSSP % gcc src/escape_parser.c -o bin/escape_parser
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 3: Successful compilation of escape sequence parser

Case 1:

```
amrutha@Amruthas-MacBook-Air OSSP % ./bin/escape_parser
myShell> echo Hello\ World

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello World

Child Process
PID = 46872

Hello World

Child process completed.
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 4: Case 1 – Handling escaped spaces as a single argument

Case 2:

```
Child process completed.
amrutha@Amruthas-MacBook-Air OSSP % ./bin/escape_parser
myShell> echo Hello\!

--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello!

Child Process
PID = 47192

Hello!

Child process completed.
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 5: Case 2 – Preserving an escaped special character

```

[amrutha@Amruthas-MacBook-Air OSSP % ./bin/escape_parser
myShell> echo My\ Project\ Folder

--- Parsed Arguments ---
Argument 1: echo
Argument 2: My Project Folder

Child Process
PID = 47541

My Project Folder

Child process completed.
amrutha@Amruthas-MacBook-Air OSSP % █

```

Figure 6: Parsing multiple escaped spaces in a complex command.

```

[amrutha@Amruthas-MacBook-Air OSSP % ./bin/escape_parser
myShell> echo Hello\
Syntax Error: Trailing escape character.
amrutha@Amruthas-MacBook-Air OSSP % █

```

Figure 7: Detection of an invalid trailing escape character

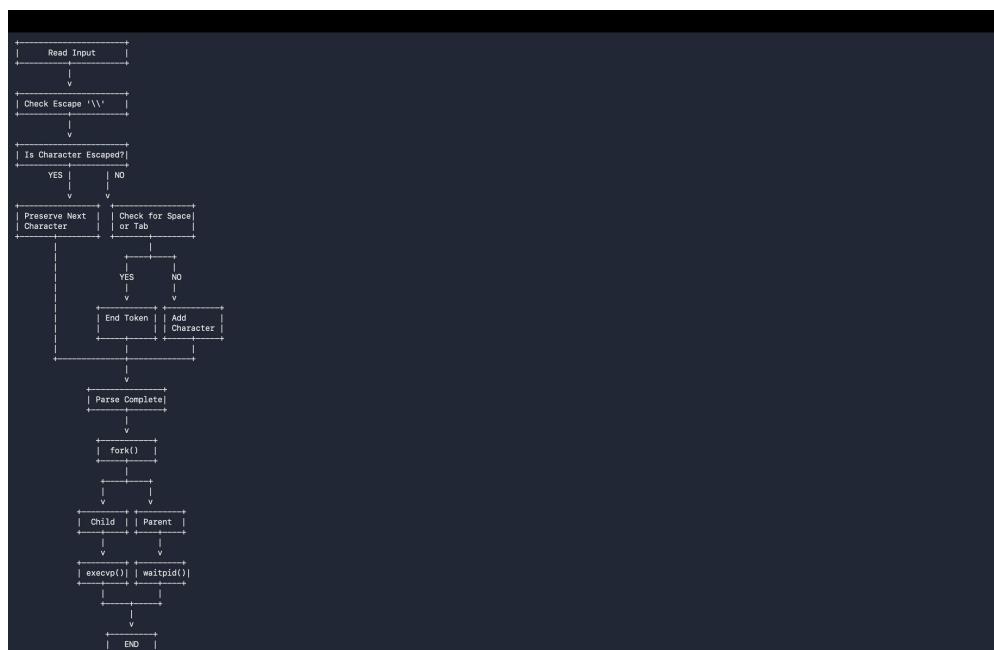


Figure 8: Terminal flowchart for escape sequence parsing and command execution

Task 5:

```
[amrutha@Amruthas-MacBook-Air ~ % cd ~/OSSP  
[amrutha@Amruthas-MacBook-Air OSSP % pwd  
/Users/amrutha/OSSP  
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 1: OSSP project directory.

```
[amrutha@Amruthas-MacBook-Air OSSP % cat src/process_launcher.c  
#include <stdio.h>  
#include <stdlib.h>  
#include <unistd.h>  
#include <sys/types.h>  
#include <sys/wait.h>  
  
int main()  
{  
    char command[100];  
    char argument[100];  
  
    printf("myshell$ ");  
  
    scanf("%99s", command);  
  
    if (scanf("%99s", argument) == 1)  
    {  
        /* Argument is provided */  
    }  
    else  
    {  
        argument[0] = '\0';  
    }  
  
    pid_t pid = fork();  
  
    if (pid < 0)  
    {  
        perror("fork failed");  
        return 1;  
    }  
}
```

Figure 2(a): Process launcher program using fork() and execvp()

```

    if (pid == 0)
    {
        printf("Child Process PID: %d\n", getpid());

        if (argument[0] != '\0')
        {
            char *args[] = {command, argument, NULL};
            execvp(command, args);
        }
        else
        {
            char *args[] = {command, NULL};
            execvp(command, args);
        }

        perror("Execution failed");
        exit(1);
    }
    else
    {
        printf("Parent Process PID: %d\n", getpid());
        printf("Created Child PID: %d\n", pid);

        int status;
        waitpid(pid, &status, 0);

        if (WIFEXITED(status))
        {
            printf("Child exited with status: %d\n",
                WEXITSTATUS(status));
        }
    }

    return 0;
}
amrutha@Amruthas-MacBook-Air OSSP % █

```

Figure 2(b): Parent process management using waitpid() and exit-status handling

Case 1:

```

amrutha@Amruthas-MacBook-Air OSSP % ./bin/process_launcher
myshell$ ls
Parent Process PID: 91668
Created Child PID: 91687
Child Process PID: 91687
assets          double_quote_flowchart.txt  File2.txt          Makefile          quote_flowchart.txt  src
bin             escape_flowchart.txt       include            obj               README.md            tests
docs            File1.txt                  LICENSE            parser_flowchart.txt  scripts
Child exited with status: 0
amrutha@Amruthas-MacBook-Air OSSP % █

```

Figure 3: Case 1 – Successful execution of the ls command using a child process

Case 2:

```
amrutha@Amruthas-MacBook-Air OSSP % ./bin/process_launcher
myshell$ ls @l
Parent Process PID: 92124
Created Child PID: 92140
Child Process PID: 92140
ls: @l: No such file or directory
Child exited with status: 1
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 4: Case 2 – Handling an invalid argument and execution error

```
amrutha@Amruthas-MacBook-Air OSSP % ./bin/process_launcher
myshell$ ls src
Parent Process PID: 92461
Created Child PID: 92477
Child Process PID: 92477
builtin.c          escape_parser.c    file_copy.c        parser.c            process_launcher.c  shell.c
double_quote_parser.c  executor.c        main.c             parser.c.save       quote_parser.c
Child exited with status: 0
amrutha@Amruthas-MacBook-Air OSSP %
```

Figure 5: Passing an argument to the child process and executing the ls command

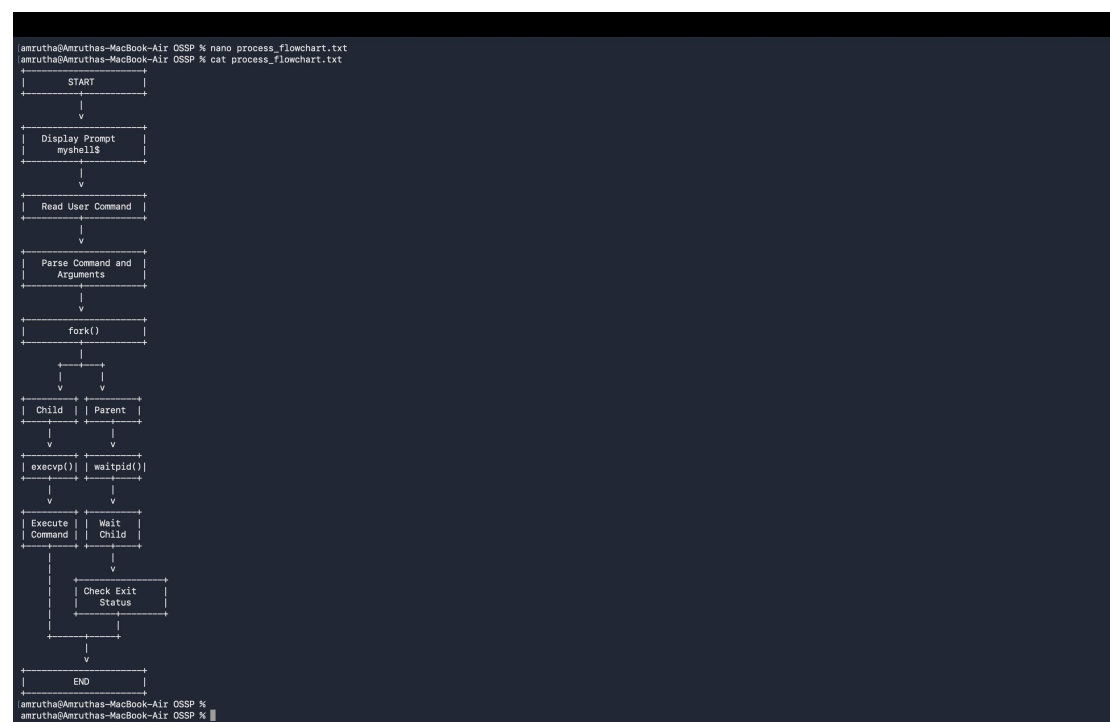


Figure 6: Terminal flowchart for process creation, command execution and parent process management

